Spring Boot Interview Notes (Detailed, Printable) + Student Management System

This is a deep, interview-ready revision sheet for **service-based + product-based** interviews. It keeps the same 60 questions, but answers are expanded with "what/why/how", examples, and common pitfalls. It also includes a complete Student Management System project (REST + JPA + Validation + Global Exception Handling).

How to use Read Part A once, then revise with "speak-able" answers. Build Part C project once yourself.

Interview trick Always mention 1 pitfall (like N+1 / LazyInitialization / transactional proxy) → interviewer feels you have real experience.

Table of Contents

Part A — Top 60 Spring Boot Interview Questions (Service + Product)

A) Core Spring + Boot Fundamentals

1) What is Spring? What problem does it solve?

Spring is a framework for building Java applications with strong support for enterprise needs. The biggest problem it solves is **tight coupling** and repetitive boilerplate code.

Before Spring: classes create dependencies using `new`. That makes code hard to test, hard to replace implementations, and error-prone.

With Spring: a container creates objects (**beans**), wires dependencies (**DI**), and manages cross-cutting concerns using **AOP**.

- **IoC/DI**: You write interfaces and business classes; Spring injects implementations.
- **AOP**: Transactions, logging, security can be applied around methods without copy-pasting.
- **Data access**: consistent exception handling and templates/abstractions.

Interview line: "Spring makes code loosely coupled, testable, and production-ready via DI, AOP, and standardized modules."

2) What is Spring Boot and why do we use it?

Spring Boot is Spring + opinionated defaults that let you build production apps quickly with minimal configuration.

It does not change Spring fundamentals; it mainly reduces setup effort and provides production tooling.

- **Auto-configuration**: sets up beans automatically based on your dependencies (classpath) and properties.
- **Starters**: dependency bundles (web, data-jpa, security) that work together (versions aligned).
- **Embedded server**: run web apps as a single jar (Tomcat/Jetty included).
- **Actuator**: health/metrics endpoints for production monitoring.
- **Externalized config**: easy environment-based configuration via properties/yml + profiles.

Example: Add `spring-boot-starter-data-jpa` + DB URL. Boot auto-creates DataSource, EntityManagerFactory, TxManager without writing config classes.

3) Explain `@SpringBootApplication`.

`@SpringBootApplication` is a convenience annotation that combines 3 powerful annotations:

- `@Configuration` → this class can define beans using `@Bean`.
- `@EnableAutoConfiguration` → activates Boot auto-configuration mechanism.
- `@ComponentScan` → scans components from this package downward.

Why it matters: If your main class is not in the root package, component scanning might miss controllers/services → you get errors like "No qualifying bean found".

Interview tip: "I keep the main class in the root package so component scanning covers all subpackages."

4) How does auto-configuration work internally?

Auto-configuration is the core Boot feature. Internally, Boot imports many "auto-config classes" listed in:

```
META-INF/spring/org.springframework.boot.autoconfigure.AutoConfigura
```

Each auto-config class uses **conditions** to decide whether to apply. Common conditions:

- `@ConditionalOnClass`: apply if a dependency class exists on classpath
- `@ConditionalOnMissingBean`: apply if user didn't define a bean
- `@ConditionalOnProperty`: apply if property enabled

Example (DataSource): If JDBC classes exist and you set datasource properties and there's no custom DataSource bean, Boot creates a DataSource bean for you.

Why this design is powerful: Boot gives defaults, but if you define your own bean, Boot backs off (thanks to `@ConditionalOnMissingBean`).

5) What happens when app starts (SpringApplication.run)?

Think of startup in phases:

1. **Environment setup**: reads config from properties/yml, env vars, args; sets profiles.
2. **Create ApplicationContext**: web apps use servlet web context.
3. **Register bean definitions**: component scan + auto-config imports register bean metadata.
4. **Instantiate singletons**: Spring creates beans, injects dependencies, runs lifecycle callbacks.
5. **Apply proxies**: for AOP (transactions, security, caching).
6. **Start embedded server**: Tomcat starts, DispatcherServlet initialized.
7. **Publish events**: ApplicationReadyEvent etc.

Interview line: "Startup builds environment, creates context, registers and instantiates beans, applies proxies, then starts the embedded server."

6) Explain IoC and DI.

IoC (Inversion of Control) means you don't control object creation; the container controls it.

DI (Dependency Injection) is the technique used: dependencies are injected into your classes.

Why it matters:

- Replace implementations easily (switch repo/service implementations)
- Unit testing becomes easy (mock dependencies)
- Loose coupling improves maintainability

Example: Controller depends on Service interface. In tests, inject a mock service. In prod, Spring injects real service.

7) Constructor injection vs field injection vs setter injection — which is best?

Constructor injection is preferred because it creates a fully-initialized object and makes dependencies explicit.

- Dependencies are mandatory → object cannot exist in invalid state
- Supports `final` fields
- Best for testing (no reflection hacks)

Setter injection is okay for optional dependencies.

Field injection is discouraged in clean code because it hides dependencies and complicates testing.

8) What is a Spring bean?

A **bean** is any object whose lifecycle is managed by the Spring container (ApplicationContext). Spring creates it, injects its dependencies, calls lifecycle callbacks, and destroys it during shutdown.

Key point: Only beans can receive DI and AOP features like `@Transactional`.

9) Bean lifecycle (high level)

Lifecycle steps you should remember:

1. Bean definition registered
2. Instantiate bean (constructor/factory)
3. Inject dependencies
4. Initialize: `@PostConstruct` / init method
5. Bean ready
6. Destroy: `@PreDestroy` / destroy method

Interview add-on: "BeanPostProcessors can modify beans before/after initialization (important for proxies)."

10) `@Component` vs `@Service` vs `@Repository` vs `@Controller` vs `@RestController`

All are **stereotype annotations** that register a class as a Spring bean, but they communicate intent:

- `@Component`: generic component
- `@Service`: service/business logic layer
- `@Repository`: data access layer; enables **exception translation**
- `@Controller`: MVC controller that returns views (HTML)
- `@RestController`: returns JSON; same as `@Controller + @ResponseBody`

Why @Repository matters: converts persistence exceptions into Spring's consistent `DataAccessException` hierarchy.

11) What is `@Autowired`? When do we use `@Qualifier` / `@Primary`?

`@Autowired` tells Spring to inject a dependency (by type). Modern Spring also supports constructor injection without `@Autowired` if there is a single constructor.

If multiple beans match the same interface:

- `@Primary`: mark one bean as default choice
- `@Qualifier("beanName")`: explicitly choose which bean to inject

Interview example: Two PaymentService implementations (UPI/Card). Use `@Qualifier` to choose.

12) Bean scopes (singleton, prototype, request, session)

Scope defines how many instances Spring creates and how long they live.

- **Singleton (default)**: one bean instance per ApplicationContext
- **Prototype**: new instance every time you request it from container
- **Request**: one instance per HTTP request (web apps)
- **Session**: one instance per HTTP session (web apps)

Important: Singleton does not mean “one per JVM globally”; it means “one per Spring container”.

13) BeanFactory vs ApplicationContext

BeanFactory is a basic container for DI. **ApplicationContext** is a richer container used in real apps.

- ApplicationContext supports **events**, **i18n**, **resource loading**, and better integration with AOP.
- It also automatically registers many infrastructure beans (post processors).

Interview point: “Spring Boot apps always use ApplicationContext.”

14) What are Spring Boot Starters?

Starters are curated dependency sets that provide a feature with compatible versions.

- `spring-boot-starter-web`: Spring MVC + embedded Tomcat + Jackson
- `spring-boot-starter-data-jpa`: Spring Data JPA + Hibernate + Tx integration
- `spring-boot-starter-test`: JUnit + Mockito + Spring Test

Why they help: you avoid dependency version conflicts and write less configuration.

15) How to disable an auto-configuration?

If you don't want Boot's default config for something, you can exclude auto-config:

```
@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)
```

Or you can override by defining your own bean (Boot backs off because of `@ConditionalOnMissingBean`).

16) What are Spring Boot conditional annotations used for auto-config?

They activate auto-config only when it makes sense:

- `@ConditionalOnClass`: dependency present
- `@ConditionalOnMissingBean`: user didn't provide custom bean
- `@ConditionalOnProperty`: feature enabled by config
- `@ConditionalOnBean`: required bean exists
- `@ConditionalOnWebApplication`: only for web contexts

B) Configuration & Properties

17) `application.properties` vs `application.yml`

Both define external configuration. YAML is hierarchical and easier for complex config.

Example (YAML advantage): nesting for datasource + security + logging is easier to read.

18) What is configuration precedence in Spring Boot?

Spring Boot resolves properties from multiple sources with precedence. Common high-to-low order:

- Command-line arguments
- Environment variables
- `application-*.properties/yml` (profile specific)
- `application.properties/yml`
- Defaults

Why asked? In production, you override configs using env vars/args, not code changes.

19) What are Profiles and why used?

Profiles let you load different configurations/beans for different environments (dev/test/prod).

- Use `application-dev.yml` for local DB and debug logs
- Use `application-prod.yml` for production DB and stricter settings

Activate via `spring.profiles.active=dev` (property or environment variable).

20) `@Value` vs `@ConfigurationProperties`

`@Value` injects a single property. It is quick but not ideal for many values.

`@ConfigurationProperties` binds a group of properties to a POJO (type-safe and cleaner), especially for config objects like external service URLs, timeouts, feature toggles.

Interview tip: Prefer `@ConfigurationProperties` for multiple related configs.

21) `@Configuration` vs `@Component`

Both are beans, but `@Configuration` is meant for configuration classes and ensures `@Bean` methods behave like singletons.

Without `@Configuration`, calling a `@Bean` method could create multiple instances in some cases. With `@Configuration`, Spring uses a proxy to return the same bean instance.

22) When do we use `@Bean`?

Use `@Bean` when:

- You need to register a third-party library class as a bean (cannot annotate it)
- You need custom creation logic (client config, custom ObjectMapper, thread pools)

Example: Define a `RestTemplate` or `WebClient` bean with timeouts.

C) REST / Web / MVC

23) Explain Spring MVC request flow

Spring MVC uses the **Front Controller** pattern. Steps:

1. Request hits **DispatcherServlet**
2. DispatcherServlet finds a controller method using HandlerMapping
3. It resolves method arguments (path variables, request body, params)
4. Controller returns a response object
5. HttpMessageConverter (Jackson) converts object ↔ JSON

Where validations happen? Before calling controller method (when using `@Valid`).

24) What is DispatcherServlet?

DispatcherServlet is Spring MVC's main servlet that routes requests to controllers and coordinates the whole request processing pipeline.

It centralizes common logic (mapping, argument resolution, exception resolution, response rendering).

25) Key web annotations you must know

- `@RestController`, `@RequestMapping`
- `@GetMapping/@PostMapping/@PutMapping/@DeleteMapping`
- `@RequestBody`, `@PathVariable`, `@RequestParam`
- `ResponseEntity`, `@ResponseStatus`
- `@CrossOrigin`

26) `@RequestParam` vs `@PathVariable`

`@PathVariable` is part of the URL path and typically identifies the resource: `/students/10`.

`@RequestParam` is used for filtering/search/paging: `/students?page=0&size=10`.

27) How do you handle exceptions globally in REST?

- `@RestControllerAdvice` for global handling
- `@ExceptionHandler` to map exceptions to responses

Return consistent error JSON with timestamp, error code, message, details.

28) How does validation work in Spring Boot?

Use Bean Validation annotations on DTO + `@Valid` in controller. On failure, Spring throws `MethodArgumentNotValidException` which you handle globally.

29) What is CORS and how do you enable it?

CORS allows browser clients from other origins to call your API. Enable using `@CrossOrigin` or global `WebMvcConfigurer` or Security config.

30) ResponseEntity vs @ResponseStatus

`@ResponseStatus` is fixed; `ResponseEntity` is dynamic and can include headers.

D) JPA / Hibernate / Spring Data JPA

31) JPA vs Hibernate vs Spring Data JPA

JPA = spec, Hibernate = implementation, Spring Data JPA = productivity layer that generates repository implementations and supports derived queries/paging.

32) What does `@Entity` do?

Marks class as a persistent JPA entity mapped to a DB table and managed by EntityManager.

33) Explain `@Id` and `@GeneratedValue`

`@Id` is primary key. `@GeneratedValue` defines strategy (IDENTITY/SEQUENCE/AUTO).

34) What is a persistence context (1st-level cache)?

EntityManager maintains managed entities, does dirty checking, and avoids duplicate DB hits for same id within a transaction.

35) Entity lifecycle states

Transient → Managed → Detached → Removed.

36) Lazy vs Eager loading

Lazy loads on access; eager loads immediately. Prefer lazy; fetch what you need via fetch-join or EntityGraph to avoid performance issues and LazyInitializationException.

37) What is N+1 problem and how to fix it?

N+1: 1 query for parents + N queries for children. Fix using fetch join / EntityGraph / batch fetching.

38) CrudRepository vs JpaRepository

JpaRepository extends CrudRepository and adds paging/sorting, flush, batch operations.

39) Derived queries in Spring Data JPA

Repository method name can generate query (findByEmail, findByNameContaining). Use @Query for complex ones.

40) JPQL vs Native SQL

JPQL uses entities/fields (portable). Native SQL uses tables/columns (DB-specific, powerful).

41) Pagination and sorting

Use Pageable and return Page<T> to provide both results and metadata (total elements/pages).

42) Cascade types & orphanRemoval (why needed)

Cascade propagates operations; orphanRemoval deletes children removed from collection. Be careful with REMOVE.

E) Transactions & Concurrency

43) What is `@Transactional`?

Defines transaction boundary via AOP proxy. Default rollback for runtime exceptions; checked exceptions need configuration.

44) Why should `@Transactional` be on service layer?

Service represents business use case and can call multiple repositories. Controller should remain thin.

45) Why `@Transactional` sometimes "doesn't work"?

Self-invocation bypasses proxy; bean not managed by Spring; checked exception; wrong proxy settings.

46) Transaction propagation types (most important)

REQUIRED joins/creates; REQUIRES_NEW always new; SUPPORTS joins if exists.

47) Isolation levels (basic)

READ_COMMITTED avoids dirty reads; REPEATABLE_READ stable row reads; SERIALIZABLE strongest but slow.

48) Optimistic vs pessimistic locking

Optimistic uses @Version; pessimistic locks rows. Choose based on conflict likelihood.

F) AOP / Filters / Interceptors

49) What is AOP and where used in Spring?

Cross-cutting concerns (transactions, logging, security, caching) applied around method calls using proxies.

50) Filter vs Interceptor vs Aspect

Filter: servlet layer; Interceptor: Spring MVC layer; Aspect: bean method layer.

G) Security Basics

51) Authentication vs Authorization

Authentication = who you are. Authorization = what you can do.

52) JWT flow in simple words

Login → server issues signed token → client sends token → server validates and authorizes by roles/claims.

H) Actuator & Production Readiness

53) What is Spring Boot Actuator?

Production endpoints: /health, /metrics, /info etc. Used for monitoring and orchestration.

54) What are readiness/liveness probes?

Liveness checks app is alive; readiness checks app can serve traffic. Actuator supports both.

I) Testing

55) `@SpringBootTest` vs `@WebMvcTest` vs `@DataJpaTest`

SpringBootTest = full context; WebMvcTest = MVC slice; DataJpaTest = repository slice.

56) `@Mock` vs `@MockBean`

@Mock for unit tests; @MockBean replaces bean in Spring context.

J) Performance & Best Practices

57) Why DTOs instead of returning Entities directly?

DTO prevents lazy loading issues, recursion, sensitive field exposure, and stabilizes API contract.

58) Caching in Spring (basic)

Enable caching and use @Cacheable/@CachePut/@CacheEvict. Redis is common in production.

59) Connection pooling (why important?)

Connections are expensive; pool reuses them. Boot uses HikariCP by default.

60) Common mistakes interviewers check

Logic in controller, returning entities, EAGER abuse, no global exception handling, missing transactions, N+1 ignored.

Part B — Important Annotations Checklist

Area	Annotations	What to say in interview
Core / DI	`@Component`, `@Service`, `@Repository`, `@Autowired`, `@Qualifier`, `@Primary`, `@Scope`	Bean creation, DI, selecting correct bean
Boot	`@SpringBootApplication` (includes config, scan, auto-config), conditional annotations	Boot scans + auto-config based on classpath/properties
Config	`@Configuration`, `@Bean`, `@Value`, `@ConfigurationProperties`, `@Profile`	Custom beans, external config, env based setup
Web	`@RestController`, `@RequestMapping`, `@GetMapping`, `@PostMapping`, `@PutMapping`, `@DeleteMapping`, `@RequestBody`, `@PathVariable`, `@RequestParam`, `@ResponseStatus`	REST mapping, binding
Validation	`@Valid`, `@NotBlank`, `@NotNull`, `@Email`, `@Size`, `@Min`, `@Max`	Validate request DTO
Exception	`@RestControllerAdvice`, `@ExceptionHandler`	Centralized error handling
JPA	`@Entity`, `@Table`, `@Id`, `@GeneratedValue`, `@Column`, relations, `@JoinColumn`, `@JoinTable`, `@Version`	ORM + locking
Tx/AOP	`@Transactional`, `@Aspect`, `@Around`	Transactions via AOP proxies
Testing	`@SpringBootTest`, `@WebMvcTest`, `@DataJpaTest`, `@MockBean`	Test slices

Part C — Student Management System Project (Full Code + Explanation)

What we build

- CRUD REST API for Students
- DTO + Validation + Global exception handling
- Spring Data JPA + H2 in-memory database (easy local run)
- Layered architecture: Controller → Service → Repository → DB

Project structure (full paths)

```
student-management/
├── pom.xml
└── src/main/
    ├── java/com/example/student/
    │   ├── StudentManagementApplication.java
    │   ├── controller/StudentController.java
    │   ├── service/StudentService.java
    │   ├── service/StudentServiceImpl.java
    │   ├── repository/StudentRepository.java
    │   ├── entity/Student.java
    │   ├── dto/StudentRequest.java
    │   ├── dto/StudentResponse.java
    │   └── exception/
    │       ├── StudentNotFoundException.java
    │       └── GlobalExceptionHandler.java
    └── resources/
        ├── application.properties
        └── data.sql (optional)
```

pom.xml

File: `pom.xml`

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://maven.apache.org/POM/4.0.0 http://m
  <modelVersion>4.0.0</modelVersion>
  <groupId>com.example</groupId>
  <artifactId>student-management</artifactId>
  <version>1.0.0</version>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
    <relativePath/>
  </parent>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-web</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>com.h2database</groupId>
      <artifactId>h2</artifactId>
      <scope>runtime</scope>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-validation</artifactId>
    </dependency>
    <dependency>
      <groupId>org.springframework.boot</groupId>
```

```
      <artifactId>spring-boot-starter-test</artifactId>
      <scope>test</scope>
    </dependency>
  </dependencies>

  <build>
    <plugins>
      <plugin>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-maven-plugin</artifactId>
      </plugin>
    </plugins>
  </build>
</project>
```

application.properties

File: `src/main/resources/application.properties`

```
spring.application.name=student-management
spring.datasource.url=jdbc:h2:mem:studentdb
spring.datasource.driverClassName=org.h2.Driver
spring.datasource.username=sa
spring.datasource.password=
spring.jpa.show-sql=true
spring.jpa.hibernate.ddl-auto=update
spring.h2.console.enabled=true
spring.h2.console.path=/h2-console
```

Optional seed data

File: `src/main/resources/data.sql`

```
INSERT INTO students (name, email, age) VALUES ('Ravi', 'ravi@gmail.com
INSERT INTO students (name, email, age) VALUES ('Neha', 'neha@gmail.com
```

Main Application

File:
`src/main/java/com/example/student/StudentManagementApplication.java`

```
package com.example.student;

import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;

@SpringBootApplication
public class StudentManagementApplication {
    public static void main(String[] args) {
        SpringApplication.run(StudentManagementApplication.class, args);
    }
}
```

Entity

File: `src/main/java/com/example/student/entity/Student.java`

```
package com.example.student.entity;

import jakarta.persistence.*;

@Entity
@Table(name = "students")
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(nullable = false)
    private String name;

    @Column(nullable = false, unique = true)
    private String email;

    private Integer age;

    public Student() {}

    public Student(Long id, String name, String email, Integer age) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Integer getAge() { return age; }
```

```
    public void setAge(Integer age) { this.age = age; }
}
```

Repository

File:

`src/main/java/com/example/student/repository/StudentRepository.java`

```
package com.example.student.repository;

import com.example.student.entity.Student;
import org.springframework.data.jpa.repository.JpaRepository;

import java.util.Optional;

public interface StudentRepository extends JpaRepository<Student, Long>
    Optional<Student> findByEmail(String email);
}
```

DTOs

File: `src/main/java/com/example/student/dto/StudentRequest.java`

```
package com.example.student.dto;

import jakarta.validation.constraints.Email;
import jakarta.validation.constraints.Min;
import jakarta.validation.constraints.NotBlank;

public class StudentRequest {

    @NotBlank(message = "Name is required")
    private String name;

    @Email(message = "Email must be valid")
    @NotBlank(message = "Email is required")
    private String email;

    @Min(value = 1, message = "Age must be greater than 0")
    private Integer age;

    public StudentRequest() {}

    public StudentRequest(String name, String email, Integer age) {
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Integer getAge() { return age; }
    public void setAge(Integer age) { this.age = age; }
}
```

File: `src/main/java/com/example/student/dto/StudentResponse.java`

```
package com.example.student.dto;

public class StudentResponse {

    private Long id;
    private String name;
    private String email;
    private Integer age;

    public StudentResponse() {}

    public StudentResponse(Long id, String name, String email, Integer
        this.id = id;
        this.name = name;
        this.email = email;
        this.age = age;
    }

    public Long getId() { return id; }
    public void setId(Long id) { this.id = id; }

    public String getName() { return name; }
    public void setName(String name) { this.name = name; }

    public String getEmail() { return email; }
    public void setEmail(String email) { this.email = email; }

    public Integer getAge() { return age; }
    public void setAge(Integer age) { this.age = age; }
}
```

Service

File: `src/main/java/com/example/student/service/StudentService.java`

```
package com.example.student.service;

import com.example.student.dto.StudentRequest;
import com.example.student.dto.StudentResponse;

import java.util.List;

public interface StudentService {
    StudentResponse createStudent(StudentRequest request);
    List<StudentResponse> getAllStudents();
    StudentResponse getStudentById(Long id);
    StudentResponse updateStudent(Long id, StudentRequest request);
    void deleteStudent(Long id);
}
```

File: `src/main/java/com/example/student/service/StudentServiceImpl.java`

```
package com.example.student.service;

import com.example.student.dto.StudentRequest;
import com.example.student.dto.StudentResponse;
import com.example.student.entity.Student;
import com.example.student.exception.StudentNotFoundException;
import com.example.student.repository.StudentRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;

import java.util.List;

@Service
public class StudentServiceImpl implements StudentService {

    private final StudentRepository studentRepository;

    public StudentServiceImpl(StudentRepository studentRepository) {
        this.studentRepository = studentRepository;
    }

    @Override
    @Transactional
    public StudentResponse createStudent(StudentRequest request) {
        studentRepository.findByEmail(request.getEmail())
                .ifPresent(s -> { throw new IllegalArgumentException("E

        Student student = new Student(null, request.getName(), request.
        Student saved = studentRepository.save(student);
        return toResponse(saved);
    }

    @Override
    @Transactional(readOnly = true)
    public List<StudentResponse> getAllStudents() {
        return studentRepository.findAll()
                .stream()
                .map(this::toResponse)
                .toList();
```

```
    }

    @Override
    @Transactional(readOnly = true)
    public StudentResponse getStudentById(Long id) {
        Student student = studentRepository.findById(id)
                .orElseThrow(() -> new StudentNotFoundException("Studen
        return toResponse(student);
    }

    @Override
    @Transactional
    public StudentResponse updateStudent(Long id, StudentRequest reques
        Student student = studentRepository.findById(id)
                .orElseThrow(() -> new StudentNotFoundException("Studen

        if (!student.getEmail().equalsIgnoreCase(request.getEmail())) {
            studentRepository.findByEmail(request.getEmail())
                    .ifPresent(s -> { throw new IllegalArgumentExceptio
        }

        student.setName(request.getName());
        student.setEmail(request.getEmail());
        student.setAge(request.getAge());

        Student saved = studentRepository.save(student);
        return toResponse(saved);
    }

    @Override
    @Transactional
    public void deleteStudent(Long id) {
        if (!studentRepository.existsById(id)) {
            throw new StudentNotFoundException("Student not found with
        }
        studentRepository.deleteById(id);
    }

    private StudentResponse toResponse(Student student) {
```

```
        return new StudentResponse(student.getId(), student.getName(),
    }
}
```

Controller

File:

`src/main/java/com/example/student/controller/StudentController.java`

```
package com.example.student.controller;

import com.example.student.dto.StudentRequest;
import com.example.student.dto.StudentResponse;
import com.example.student.service.StudentService;
import jakarta.validation.Valid;
import org.springframework.http.HttpStatus;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/api/students")
public class StudentController {

    private final StudentService studentService;

    public StudentController(StudentService studentService) {
        this.studentService = studentService;
    }

    @PostMapping
    @ResponseStatus(HttpStatus.CREATED)
    public StudentResponse createStudent(@Valid @RequestBody StudentReq
        return studentService.createStudent(request);
    }

    @GetMapping
    public List<StudentResponse> getAllStudents() {
        return studentService.getAllStudents();
    }

    @GetMapping("/{id}")
    public StudentResponse getStudentById(@PathVariable Long id) {
        return studentService.getStudentById(id);
    }

    @PutMapping("/{id}")
    public StudentResponse updateStudent(@PathVariable Long id, @Valid
```

```
        return studentService.updateStudent(id, request);
    }

    @DeleteMapping("/{id}")
    @ResponseStatus(HttpStatus.NO_CONTENT)
    public void deleteStudent(@PathVariable Long id) {
        studentService.deleteStudent(id);
    }
}
```

Exceptions

File:

`src/main/java/com/example/student/exception/StudentNotFoundException.java`

```
package com.example.student.exception;

public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

File:

`src/main/java/com/example/student/exception/GlobalExceptionHandler.java`

```
package com.example.student.exception;

import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.MethodArgumentNotValidException;
import org.springframework.web.bind.annotation.ExceptionHandler;
import org.springframework.web.bind.annotation.RestControllerAdvice;

import java.time.Instant;
import java.util.HashMap;
import java.util.Map;

@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<Map<String, Object>> handleStudentNotFound(St
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", Instant.now().toString());
        body.put("error", "STUDENT_NOT_FOUND");
        body.put("message", ex.getMessage());
        return ResponseEntity.status(HttpStatus.NOT_FOUND).body(body);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<Map<String, Object>> handleValidation(MethodA
        Map<String, Object> fieldErrors = new HashMap<>();
        ex.getBindingResult().getFieldErrors()
                .forEach(err -> fieldErrors.put(err.getField(), err.get

        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", Instant.now().toString());
        body.put("error", "VALIDATION_FAILED");
        body.put("details", fieldErrors);

        return ResponseEntity.badRequest().body(body);
    }

    @ExceptionHandler(IllegalArgumentException.class)
```

```
    public ResponseEntity<Map<String, Object>> handleBadRequest(Illegal
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", Instant.now().toString());
        body.put("error", "BAD_REQUEST");
        body.put("message", ex.getMessage());
        return ResponseEntity.badRequest().body(body);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<Map<String, Object>> handleGeneric(Exception
        Map<String, Object> body = new HashMap<>();
        body.put("timestamp", Instant.now().toString());
        body.put("error", "INTERNAL_ERROR");
        body.put("message", "Something went wrong");
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR).
    }
}
```

How to run & test

- Run: `mvn spring-boot:run`
- API base: `http://localhost:8080/api/students`
- H2 Console: `http://localhost:8080/h2-console` (JDBC URL: `jdbc:h2:mem:studentdb`)

End of detailed printable notes.